Техническое задание

Проект: AntiSpam Guardian

Формат: Telegram SaaS anti-spam bot

---

1. Общая идея проекта

Нужно развивать Telegram-бота AntiSpam Guardian как SaaS-продукт для защиты Telegram-групп от спама.

Бот уже частично реализован и умеет:

- подключаться к группам
- модерировать сообщения
- удалять ссылки
- работать с частью панели
- сохранять чаты и настройки в базу
- поддерживать лог-чаты
- работать через Docker / VPS / GitHub

Цель — не просто “антиспам-бот”, а продукт с панелью управления, несколькими чатами на пользователя, подпиской, лог-чатами и дальнейшей монетизацией.

---

2. Что уже есть в проекте

На текущий момент в проекте уже есть базовая структура и часть функционала.

Уже реализовано

2.1. Базовая архитектура

Есть разделение по файлам:

- app/main.py
- app/db/models.py
- app/db/session.py
- app/handlers/start.py
- app/handlers/onboarding.py
- app/handlers/moderation.py
- app/handlers/panel_dm.py
- app/handlers/log_setup.py
- app/handlers/log_actions.py
- app/handlers/health.py
- app/handlers/whitelist.py
- app/handlers/stopwords.py
- app/utils/admins.py

2.2. База данных

Уже есть модели для:

- чатов
- правил
- whitelist
- stopwords
- менеджеров чатов / владельцев

2.3. Онбординг

Есть /start, /check, базовое подключение чата.

2.4. Модерация

Есть ядро moderation pipeline:

- links
- mentions
- stopwords
- anti-edit
- newbie mode
- action mode delete / mute / ban
- log keyboard
- logging

2.5. Лог-чаты

Есть логика log_setup.py, есть привязка лог-группы.

2.6. Панель

Есть panel DM с выбором чата, настройками и частью кнопок.

2.7. Технический деплой

Есть:

- запуск на локальной машине
- nohup
- Docker
- GitHub
- git pull / restart

---

3. Основная проблема текущей версии

Сейчас бот уже рабочий, но UX и архитектура ещё “сырая”.

Основные проблемы

3.1. Панель слишком плоская

На главном экране слишком много кнопок. Настройки перегружают интерфейс.

3.2. Смешение сущностей

Сейчас путаются:

- защищаемые чаты
- лог-чаты
- выбранный текущий чат

3.3. Нет SaaS-слоя

Нет полноценной сущности пользователя как клиента продукта:

- нет подписки
- нет лимитов чатов
- нет истории оплаты
- нет тарифов

3.4. Нет законченной модели монетизации

Нужно не просто “бот с функциями”, а платный продукт.

---

4. Цель новой версии

Нужно превратить текущего бота в понятный SaaS anti-spam продукт со следующими сущностями:

- пользователь
- его подписка

- его подключённые чаты
- лог-чаты
- правила модерации для каждого чата
- панель управления
- тарифы
- оплата
- виральность / рост

---

5. Продуктовая модель

5.1. Бизнес-модель

Монетизация строится по количеству подключённых чатов.

Не по функциям отдельно, а по лимиту чатов.

Функции могут быть частично ограничены по тарифу, но основа монетизации — это сколько чатов под защитой.

5.2. Тарифы

FREE

- 1 защищаемый чат
- базовые фильтры
- без или с ограниченными логами
- без anti-raid
- без расширенной автоматизации

PRO

- до 5 чатов
- все базовые функции
- лог-чаты
- anti-edit
- stopwords
- whitelist
- кастомизация наказаний
- anti-raid / защита от нашествия ботов

BUSINESS

- до 20 или 50 чатов
- всё из PRO
- расширенные лимиты
- приоритет
- возможно будущая аналитика / расширенные отчёты

5.3. Подписка

Подписка оплачивается по времени:

- 1 месяц
- 3 месяца
- 6 месяцев
- 12 месяцев
- 24 месяца

Цена за месяц уменьшается при оплате длинного периода.

---

6. Образ и позиционирование бота

Бот не должен быть “сухим сервисом”.

Нужен характер:

AntiSpam Guardian = страж / защитник чата

Стиль:

- жёсткий
- уверенный
- анти-спамерский
- слегка брутальный
- но без дешёвого кринжа и без мата

6.1. Примеры сообщений в чат

Вместо сухого:

message deleted

лучше:

- ⚔️ Guardian уничтожил спам
- 🚫 Ссылка ликвидирована
- 🛡️ Guardian держит чат под защитой
- 🚨 Включён режим шторма
- 😈 Спамер отправлен в мусор

Нужно предусмотреть режимы видимости:

- тихий режим
- умеренный режим
- публичный режим

6.2. Guardian Levels

В перспективе можно добавить уровни бота:

- Guardian Level 1
- Guardian Level 2
- Guardian Level 3

Рост уровня — по количеству успешно предотвращённого спама.

Это не критично для MVP, но важно как продуктовая идея.

---

7. Архитектура сущностей

7.1. User

Нужна полноценная модель пользователя SaaS, даже если сейчас часть логики держится просто на Telegram user id.

Пример полей

```
id
telegram_id
username
first_name
created_at
subscription_until
tariff
chat_limit
is_admin
status
```

Для чего

- хранить подписку
- считать лимит чатов
- делать оплату
- делать напоминания
- делать скидки
- делать воронку onboarding

7.2. Chat

Защищаемый чат.

Поля

```
id
title
owner_user_id
is_active
log_chat_id
created_at
is_log_chat
```

Важно

Нужно разделить:

- защищаемые чаты
- лог-чаты

Они не должны путаться в панели.

Можно решить либо отдельной таблицей лог-чатов, либо полем is_log_chat.

7.3. Rule

Настройки конкретного защищаемого чата.

Поля

```
chat_id
filter_links
filter_mentions
filter_stopwords
action_mode
mute_minutes
anti_edit
newbie_enabled
newbie_minutes
log_enabled
raid_enabled
raid_threshold
raid_window_seconds
raid_action
public_messages_mode
```

7.4. StopWord

Список стоп-слов по каждому чату.

7.5. WhitelistUser / WhitelistDomain

Белые списки пользователей и доменов.

7.6. Payment

Таблица оплат.

Поля

```
id
user_id
amount
currency
months
tariff
created_at
expires_at
status
provider
payment_id
```

---

8. Подключение чатов

8.1. Защищаемый чат

Сценарий:

1. Пользователь добавляет бота в группу
2. Даёт права администратора
3. В группе вводит /check
4. Бот проверяет:
    - это группа / супергруппа
    - вызывающий пользователь админ
    - бот админ
    - у бота есть право удалять сообщения
5. Бот сохраняет чат в БД
6. Создаёт Rule по умолчанию
7. Отправляет подтверждение и кнопку панели

8.2. Лог-чат

Сценарий:

1. Пользователь создаёт отдельную группу под отчёты

2. Добавляет туда бота
3. Даёт боту права
4. В этой группе вводит /setlog
5. Группа сохраняется как доступный лог-чат
6. Далее в панели можно выбрать этот лог-чат для конкретного защищаемого чата

---

9. Панель управления

9.1. Главный принцип

Главная панель должна быть короткой и продуктовой, а не “свалкой всех кнопок”.

Главный экран

Показывает:

- текущий выбранный защищаемый чат
- сколько чатов подключено
- какой тариф
- до какого числа подписка
- какой лог-чат назначен текущему чату

Кнопки главного меню

- 🛡️ Защита
- 🧾 Отчёты
- 💬 Чаты
- 💳 Тариф и оплата
- ➕ Подключить чат

9.2. Вложенное меню защиты

Очень важно:

Фильтры, Наказания, Новички, Стоп-слова должны быть не в главном меню, а под одной общей кнопкой.

Структура

```
🛡️ Защита
 ├─ ⚙️ Фильтры
 ├─ 🔨 Наказания
 ├─ 👶 Новички
 ├─ 🧠 Стоп-слова
 └─ 🚨 Анти-рейд
```

Это обязательно.

9.3. Фильтры

Подраздел:

- ссылки
- упоминания
- anti-edit
- возможно доменные whitelist-настройки

9.4. Наказания

Подраздел:

- режим delete / mute / ban
- длительность mute
- отдельная настройка наказания для разных сценариев в будущем

9.5. Новички

Подраздел:

- режим новичков ВКЛ / ВЫКЛ
- окно новичка (в минутах)
- логика усиленного контроля для новых участников

9.6. Стоп-слова

Подраздел:

- добавить
- удалить
- показать список

9.7. Анти-рейд

Подраздел:

- ВКЛ / ВЫКЛ
- порог срабатывания (например N новых участников)
- окно времени
- действие при рейде
- включать ли строгий режим
- включать ли запрет сообщений новичкам

Эта функция может быть PRO / BUSINESS.

9.8. Чаты

Раздел:

- подключённые чаты
- сменить текущий чат
- лог-чаты

Важно

Нельзя смешивать:

- защищаемые чаты
- лог-чаты

9.9. Отчёты

Раздел:

- включить / выключить отчёты
- выбрать лог-чат
- показать как работает

В “как работает”

Нужно текстом объяснить:

1. создать группу логов
2. добавить бота
3. дать права
4. в этой группе написать /setlog
5. вернуться в панель и выбрать лог-чат

9.10. Тариф и оплата

Раздел:

- текущий тариф
- лимит чатов
- дата окончания
- выбрать период оплаты
- оплатить

---

10. Модерация

10.1. Общая логика

Нужно оставить pipeline-подход:

```
evaluate() -> apply_action() -> send_log()
```

Это уже хорошая архитектура.

10.2. Что должно проверяться

- ссылки
- упоминания
- стоп-слова
- anti-edit
- режим новичков
- anti-raid
- whitelist users
- whitelist domains
- skip admins

10.3. Подводные камни moderation

Команды не должны перехватываться moderation раньше onboarding/start

Уже была проблема с порядком роутеров.

Нужно явно следить, чтобы:

- команды /check, /start, /health, /setlog не съедались moderation-хендлером

Упоминания

Нужно учитывать:

- @username
- text_mention
- mention entities

Anti-edit

Сам факт редактирования не должен быть нарушением.

Нарушением должно быть:

- сообщение, которое после редактирования стало нарушающим правила

Newbie mode

Сейчас уже есть TTL/LRU-подход.

Важно не ломать его, чтобы память не росла бесконечно.

Кэш

Уже есть кэш для stopwords / whitelist.

Нужно сохранять bounded memory подход, чтобы бот не распухал на сотнях чатов.

---

11. Логи и действия по ним

11.1. Что логировать

- причина
- действие
- пользователь
- чат
- исходный текст (обрезанный)
- было ли удаление успешным
- было ли наказание успешным

11.2. Inline действия в логах

Оставить и развивать:

- Разбанить
- Размутить

Через reply / inline callbacks.

---

12. Product-led growth / виральность

Нужно заложить механики роста.

12.1. Публичный след бота

В публичном или умеренном режиме бот может оставлять короткие сообщения в чат:

- о бане
- о рейде
- о спаме

Это помогает виральности.

12.2. Кнопка “Добавить в другой чат”

После успешного /check можно предлагать:

- добавить бота в другой чат
- открыть панель

12.3. Кнопка “Поделиться ботом”

Можно добавить в будущем:

- шаблон сообщения
- переход к боту

12.4. Статистика

В будущем:

- “За неделю бот удалил 124 сообщения”
- “Предотвращено 3 рейда”

Это можно использовать и в панели, и для шаринга.

---

13. Воронка и напоминания

Это не обязательно для MVP, но нужно заложить архитектурно.

13.1. Напоминания

Если пользователь начал подключение, но не завершил:

- через 12/24 часа отправить напоминание

Пример:

- “Осталось подключить чат”
- “Защита почти готова”

13.2. Квалификационные вопросы

В будущем можно сделать onboarding-вопросы:

- сколько чатов нужно защитить
- какого размера чаты
- нужен ли лог-чат
- нужна ли защита от рейдов

И на основе этого предлагать тариф.

---

14. Оплата

14.1. Не обязательно делать сразу

Но архитектура под оплату должна быть уже сейчас.

14.2. Варианты оплаты

В будущем возможны:

- Telegram Payments
- CryptoBot
- ручная оплата / перевод
- YooKassa / Stripe / др.

14.3. Что важно сейчас

Даже если платёжки не делать сразу, в базе уже должны быть:

- user

- tariff
- subscription_until
- chat_limit
- payments

---

15. Подводные камни и пожелания

15.1. Не смешивать protected chats и log chats

Это один из ключевых UX-багов текущей версии.

15.2. Не делать слишком плоское меню

Главное меню должно быть коротким.

15.3. Не превращать всё в один файл

Текущая структура по файлам правильная. Можно укрупнять, но не сливать всё в monolith-файл.

15.4. Не ломать bounded cache подход

Уже сделаны LRU/TTL идеи — важно сохранить масштабируемость.

15.5. Проверять реальный running code

В проекте уже были кейсы, когда запускался старый процесс. Нужен понятный deploy/restart flow.

---

16. Что делать сейчас

Этап 1 — ближайший

Сделать и довести до стабильного состояния:

- корректный /check
- корректный /setlog
- правильную логику панели
- раздельные protected chats / log chats
- вложенное меню:
  - Фильтры
  - Наказания
  - Новички
  - Стоп-слова
- корректный выбор текущего чата
- корректный выбор лог-чата
- тарифы по количеству чатов
- базовую модель users + subscription + chat_limit

---

17. Что делать потом

Этап 2

- оплата
- продление подписки
- анти-рейд

- логика истечения подписки
- статистика

Этап 3

- напоминания
- onboarding funnel
- уровни Guardian
- публичные сообщения / виральность
- share / invite mechanics

---

18. Примеры кода, которые желательно сохранить по стилю

moderation pipeline

```
async def pipeline(message: Message, *, edited: bool = False) -> None:
    async with await get_session() as session:
        v = await evaluate(session, message, edited=edited)
        if not v.should_act:
            return

        ok_action, action_label, deleted_ok = await apply_action(message,
v)

        await send_log(
            session,
            message,
            v,
            action_label=action_label,
            ok_action=ok_action,
            deleted_ok=deleted_ok,
        )
```

bounded cache idea

```
PANEL_CACHE: "OrderedDict[int, Tuple[int, datetime]]" = OrderedDict()
CACHE_MAX = 3000
CACHE_TTL = timedelta(days=3)
```

chat ownership idea

```
select(Chat).where(
    Chat.is_active == True,
    or_(
        Chat.owner_user_id == user_id,
        Chat.id.in_(select(sub.c.chat_id))
    )
)
```

Эти идеи хорошие, их лучше не ломать, а развивать.

---

19. Ключевая продуктовая идея

Бот — это не просто технический антиспам.

Это Guardian, цифровой страж чата.

Продукт должен сочетать:

- жёсткую модерацию
- понятную панель
- SaaS-модель
- характер
- виральность
- рост через контент и рекомендации